

INCOEX Especialización Ciencia de Datos
Proyecto Final Modulo 02
Manejo y preparación de datos



Profesor
Alonso Berman Moya Umaña

Estudiante
Cristhian Josué Córdoba Campos

Manejo de datos con SQL SERVER

2026

Diagrama Modelo Físico**3****Scripts****3**

Creación de la base de datos matricula

3

Insertar datos

6

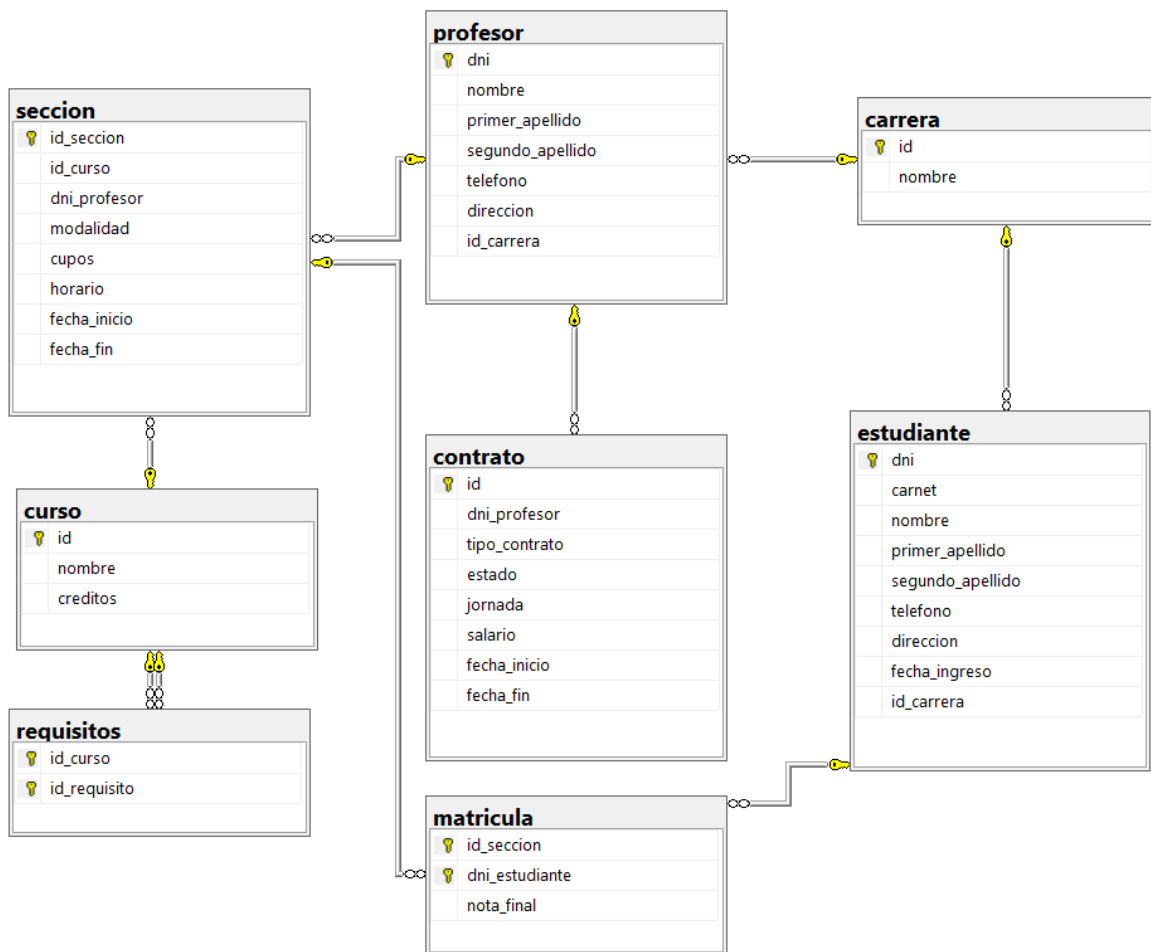
Consultas

10

Vistas

15

Diagrama Modelo Físico



Scripts

Creación de la base de datos matricula

```

-- Creacion e inicializacion de base de datos
USE master;
DROP DATABASE matricula

CREATE DATABASE matricula
GO

USE matricula;
GO

/* Tabla: carrera
Registra las carreras existentes
*/
CREATE TABLE carrera (
    id CHAR(2) PRIMARY KEY, -- 'TI', 'AD', 'ME'...

```

```

        nombre VARCHAR(50) NOT NULL
    );

/*
Tabla: estudiante
Registra información personal y académica de los alumnos.
*/
CREATE TABLE estudiante(
dni CHAR(9) PRIMARY KEY CHECK (dni NOT LIKE '%[^0-9]%), -- Garantizar formato de cédula
nacional
carnet CHAR(6) UNIQUE, -- Código institucional único (Ej: C12X15)
nombre VARCHAR(30) NOT NULL, -- Nombre del alumno
primer_apellido VARCHAR(30) NOT NULL, -- Primer apellido obligatorio
segundo_apellido VARCHAR(30), -- Segundo apellido opcional
telefono CHAR(8) CHECK (telefono NOT LIKE '%[^0-9]%), -- Formato de celular local
direccion VARCHAR(255), -- Dirección de vivienda
fecha_ingreso DATE NOT NULL, -- Fecha de incorporación a la institución
id_carrera CHAR(2) REFERENCES carrera(id)
);
GO

/*
Tabla: profesor
Registra información personal de los docentes de la institución
*/
CREATE TABLE profesor(
dni CHAR(9) PRIMARY KEY CHECK (dni NOT LIKE '%[^0-9]%), -- Garantizar formato de cédula
nacional
nombre VARCHAR(30) NOT NULL, -- Nombre del alumno
primer_apellido VARCHAR(30) NOT NULL, -- Primer apellido obligatorio
segundo_apellido VARCHAR(30), -- Segundo apellido opcional
telefono CHAR(8) CHECK (telefono NOT LIKE '%[^0-9]%), -- Formato de celular local
direccion VARCHAR(255), -- Dirección de vivienda
id_carrera CHAR(2) REFERENCES carrera(id)
);
GO

/*
Tabla contrato
Historico de contratos laborales asignados a los profesores.
*/
CREATE TABLE contrato(
id INT PRIMARY KEY IDENTITY (1,1), -- Identificador autoincremental
dni_profesor CHAR(9) REFERENCES profesor(dni) NOT NULL, -- Referencia al profesor
tipo_contrato VARCHAR(50) NOT NULL CHECK (tipo_contrato IN
('tiempo completo', 'medio tiempo', 'por horas', 'cátedra')), -- Modalidades
permitidas
estado VARCHAR(40) NOT NULL CHECK (estado IN
('activo', 'finalizado', 'suspendido')), -- Estado de la relación laboral
jornada tinyint NOT NULL CHECK (jornada BETWEEN 0 AND 48), -- Carga horaria semanal
salario DECIMAL(10,2) NOT NULL CHECK (salario >= 0), -- Monto salarial
fecha_inicio DATE NOT NULL, -- Inicio de vigencia del contrato
fecha_fin DATE,
CONSTRAINT CHK_fechas CHECK (fecha_fin IS NULL OR fecha_fin >= fecha_inicio) -- Fin de
vigencia

```

```

);
GO

/*
Tabla curso
Catálogo de materias que ofrece la institución
*/
CREATE TABLE curso(
id CHAR(6) PRIMARY KEY, -- Formato esperado: 2 letras de dpto + 4 números (Ej: MA1001)
nombre VARCHAR(127) NOT NULL, -- Nombre de la asignatura
creditos TINYINT NOT NULL CHECK (creditos BETWEEN 0 AND 20) -- Valor académico (0 a 20)
);
GO

/*
Tabla requisito
Mapea relación N:M para los prerrequisitos entre cursos
*/
CREATE TABLE requisitos (
id_curso CHAR(6) REFERENCES curso(id),
id_requisito CHAR(6) REFERENCES curso(id)
PRIMARY KEY (id_curso, id_requisito),
CONSTRAINT chk_requisito_diferente CHECK (id_requisito <> id_curso) -- El CHECK definido
a nivel de tabla sí puede comparar dos columnas
);
GO

/*
Tabla seccion
Ofertas para impartir cursos en un ciclo.
*/
CREATE TABLE seccion(
id_seccion INT PRIMARY KEY IDENTITY (1,1), -- Identificador único
id_curso CHAR(6) NOT NULL REFERENCES curso(id), -- Cursos al que pertenece
dni_profesor CHAR(9) REFERENCES profesor(dni), -- Profesor asignado
modalidad VARCHAR(127) NOT NULL CHECK (modalidad IN
('Virtual', 'Presencial', 'Bimodal')), -- Modalidad de dictado
cupos TINYINT NOT NULL CHECK (cupos BETWEEN 1 AND 60), -- Limite máximo de alumnos por
sección
horario VARCHAR(511) NOT NULL, -- Días y horas de impartición (Texto estructurado)
fecha_inicio DATE NOT NULL, -- Inicio del periodo lectivo
fecha_fin DATE NOT NULL -- Cierre del periodo lectivo
);
GO

/*
Tabla matricula
Registrar la inscripción de un estudiante en una sección
Resuelve relación N:M entre estudiante y seccion
*/
CREATE TABLE matricula(
id_seccion INT REFERENCES seccion(id_seccion), -- Seccion inscrita
dni_estudiante CHAR(9) REFERENCES estudiante(dni), -- Estudiante matriculado
nota_final TINYINT CHECK (nota_final BETWEEN 0 AND 100) -- Calificación obtenida
PRIMARY KEY(id_seccion, dni_estudiante)
);

```

```
GO
```

Insertar datos

```
USE matricula
GO

INSERT INTO carrera (id, nombre) VALUES
('TI', 'Ingeniería de Sistemas'), ('AD', 'Administración'), ('II', 'Ingeniería
Industrial'),
('DE', 'Derecho'), ('PS', 'Psicología'), ('ME', 'Medicina'),
('AR', 'Arquitectura'), ('EC', 'Economía'), ('BI', 'Biología'),
('EN', 'Enfermería'), ('SO', 'Sociología'), ('QU', 'Química'),
('ED', 'Educación'), ('CO', 'Contabilidad'), ('AG', 'Agronomía');
GO

SET NOCOUNT ON;

DECLARE @i INT = 1;
DECLARE @dni CHAR(9), @carnet CHAR(6), @prov INT, @id_car CHAR(2);
DECLARE @f_ingreso DATE, @dir VARCHAR(255);

-- Listas para aleatoriedad
DECLARE @Nombres TABLE (id INT IDENTITY, n VARCHAR(30));
INSERT INTO @Nombres VALUES
('Juan'),('Maria'),('Luis'),('Ana'),('Carlos'),('Elena'),('Jose'),('Laura'),('Pedro'),('
Lucia'),('Diego'),('Sofia'),('Andres'),('Valeria'),('Jorge');
DECLARE @Apellidos TABLE (id INT IDENTITY, a VARCHAR(30));
INSERT INTO @Apellidos VALUES
('Rodriguez'),('Gonzalez'),('Morales'),('Vargas'),('Castillo'),('Jimenez'),('Chaves'),('
Mora'),('Solis'),('Pereira'),('Castro'),('Guzman'),('Rojas'),('Salazar'),('Hidalgo');

WHILE @i <= 2000
BEGIN
    SET @prov = (ABS(CHECKSUM(NEWID())) % 7) + 1;
    SET @id_car = (SELECT TOP 1 id FROM carrera ORDER BY NEWID());
    SET @dni = CAST(@prov AS CHAR(1)) + RIGHT('00000000' + CAST(ABS(CHECKSUM(NEWID())) %
100000000 AS VARCHAR(8)), 8);
    SET @carnet = LEFT(@id_car, 1) + RIGHT('00000' + CAST(@i AS VARCHAR(5)), 5);

    SET @dir = CASE @prov
        WHEN 1 THEN 'San Jose' WHEN 2 THEN 'Alajuela' WHEN 3 THEN 'Cartago' WHEN 4 THEN
'Heredia'
        WHEN 5 THEN 'Guanacaste' WHEN 6 THEN 'Puntarenas' WHEN 7 THEN 'Limon' END + ', '
+
        (SELECT TOP 1 a FROM @Apellidos ORDER BY NEWID()) + ' central';

    SET @f_ingreso = DATEADD(DAY, ABS(CHECKSUM(NEWID())) % 1825, '2020-01-01');

    IF NOT EXISTS (SELECT 1 FROM estudiante WHERE dni = @dni OR carnet = @carnet)
    BEGIN
        INSERT INTO estudiante (dni, carnet, nombre, primer_apellido, segundo_apellido,
telefono, direccion, fecha_ingreso, id_carrera)
```

```

VALUES (
    @dni, @carnet,
    (SELECT TOP 1 n FROM @Nombres ORDER BY NEWID()),
    (SELECT TOP 1 a FROM @Apellidos ORDER BY NEWID()),
    (SELECT TOP 1 a FROM @Apellidos ORDER BY NEWID()),
    -- Lógica corregida: Prefijo (8,7,6) + 7 dígitos exactos con ceros a la
izquierda
    CAST((ABS(CHECKSUM(NEWID())) % 3 + 6) AS CHAR(1)) +
    RIGHT('000000' + CAST(ABS(CHECKSUM(NEWID())) % 1000000 AS VARCHAR(7)), 7),
    @dir, @f_ingreso, @id_car
);
SET @i = @i + 1;
END
END
PRINT '2000 Estudiantes insertados con éxito.';
GO

SET NOCOUNT ON;
DECLARE @j INT = 1;
DECLARE @dni_p CHAR(9), @id_car_p CHAR(2), @prov_p INT;

WHILE @j <= 400
BEGIN
    SET @prov_p = (ABS(CHECKSUM(NEWID())) % 7) + 1;
    -- Asegurar que cada carrera tenga profesores, luego azar
    IF @j <= 15
        SET @id_car_p = (SELECT id FROM (SELECT id, ROW_NUMBER() OVER (ORDER BY id) as
rn FROM carrera) t WHERE rn = @j);
    ELSE
        SET @id_car_p = (SELECT TOP 1 id FROM carrera ORDER BY NEWID());

    SET @dni_p = CAST(@prov_p AS CHAR(1)) + RIGHT('00000000' +
CAST(ABS(CHECKSUM(NEWID())) % 100000000 AS VARCHAR(8)), 8);

    IF NOT EXISTS (SELECT 1 FROM profesor WHERE dni = @dni_p)
    BEGIN
        INSERT INTO profesor (dni, nombre, primer_apellido, segundo_apellido, telefono,
direccion, id_carrera)
        VALUES (
            @dni_p,
            (SELECT TOP 1 n FROM (VALUES
('Carlos'),('Marta'),('Roberto'),('Ligia'),('Hernan'),('Sonia'),('Ricardo')) AS T(n)
ORDER BY NEWID()),
            (SELECT TOP 1 a FROM (VALUES
('Vargas'),('Jimenez'),('Mora'),('Soto'),('Rojas')) AS A(a) ORDER BY NEWID()),
            (SELECT TOP 1 a FROM (VALUES ('Castro'),('Perez'),('Solano'),('Gomez')) AS
A2(a) ORDER BY NEWID()),
            -- Lógica corregida: Prefijo 2 o 4 + 7 dígitos exactos
            CAST((ABS(CHECKSUM(NEWID())) % 2 * 2 + 2) AS CHAR(1)) +
            RIGHT('0000000' + CAST(ABS(CHECKSUM(NEWID())) % 10000000 AS VARCHAR(7)), 7),
            (CASE @prov_p WHEN 1 THEN 'San Jose' WHEN 2 THEN 'Alajuela' WHEN 3 THEN
'Cartago' WHEN 4 THEN 'Heredia'
            WHEN 5 THEN 'Guanacaste' WHEN 6 THEN 'Puntarenas' WHEN 7 THEN 'Limon' END),
            @id_car_p
        );
        SET @j = @j + 1;
    END
END

```

```

        END
    END
    PRINT '400 Profesores insertados con éxito.';
    GO

    USE matricula;
    GO
    SET NOCOUNT ON;

    DECLARE @dni_p CHAR(9), @tipo VARCHAR(50), @estado VARCHAR(40), @salario DECIMAL(10,2),
    @f_ini DATE;

    DECLARE prof_cursor CURSOR FOR SELECT dni FROM profesor;
    OPEN prof_cursor;
    FETCH NEXT FROM prof_cursor INTO @dni_p;

    WHILE @@FETCH_STATUS = 0
    BEGIN
        -- 1. Decidir si el profesor tiene un contrato viejo finalizado (40% de
        probabilidad)
        IF (ABS(CHECKSUM(NEWID()))) % 10 < 4
        BEGIN
            SET @f_ini = DATEADD(YEAR, -3, '2020-01-01');
            INSERT INTO contrato (dni_profesor, tipo_contrato, estado, jornada, salario,
            fecha_inicio, fecha_fin)
            VALUES (@dni_p, 'medio tiempo', 'finalizado', 20, 450000, @f_ini, DATEADD(YEAR,
            2, @f_ini));

            -- Contrato actual activo
            INSERT INTO contrato (dni_profesor, tipo_contrato, estado, jornada, salario,
            fecha_inicio, fecha_fin)
            VALUES (@dni_p, 'tiempo completo', 'activo', 40, 950000, DATEADD(DAY, 1,
            DATEADD(YEAR, 2, @f_ini)), NULL);
        END
        ELSE
        BEGIN
            -- Solo contrato activo desde su ingreso
            SET @tipo = (SELECT TOP 1 val FROM (VALUES ('tiempo completo'),('medio
            tiempo'),('por horas'),('cátedra')) AS T(val) ORDER BY NEWID());
            SET @salario = CASE @tipo WHEN 'tiempo completo' THEN 1000000 WHEN 'medio
            tiempo' THEN 500000 ELSE 250000 END;
            INSERT INTO contrato (dni_profesor, tipo_contrato, estado, jornada, salario,
            fecha_inicio, fecha_fin)
            VALUES (@dni_p, @tipo, 'activo', CASE WHEN @tipo='tiempo completo' THEN 40 ELSE
            20 END, @salario, '2020-01-01', NULL);
        END

        FETCH NEXT FROM prof_cursor INTO @dni_p;
    END
    CLOSE prof_cursor; DEALLOCATE prof_cursor;
    PRINT '400+ Contratos generados.';
    GO

    SET NOCOUNT ON;
    DECLARE @cod_car CHAR(2), @i INT, @nivel INT, @curso_id CHAR(6), @nom_car VARCHAR(100);

```



```

DECLARE car_cursor CURSOR FOR SELECT id, nombre FROM carrera;
OPEN car_cursor;
FETCH NEXT FROM car_cursor INTO @cod_car, @nom_car;

WHILE @@FETCH_STATUS = 0
BEGIN
    SET @i = 1;
    WHILE @i <= 50
    BEGIN
        SET @nivel = CASE WHEN @i <= 12 THEN 1000 WHEN @i <= 25 THEN 2000 WHEN @i <= 38
        THEN 3000 ELSE 4000 END;
        SET @curso_id = @cod_car + CAST(@nivel + @i AS CHAR(4));

        INSERT INTO curso (id, nombre, credits)
        VALUES (@curso_id, @nom_car + ' Lvl ' + CAST(@i AS VARCHAR),
        (ABS(CHECKSUM(NEWID())) % 4) + 2);

        -- Requisito lógico: Si es nivel 2000+, pide un curso de nivel inferior de la
        misma carrera
        IF @nivel > 1000
        BEGIN
            DECLARE @req CHAR(6) = (SELECT TOP 1 id FROM curso WHERE id LIKE @cod_car +
            '%' AND id < @curso_id ORDER BY id DESC);
            IF @req IS NOT NULL
                INSERT INTO requisitos (id_curso, id_requisito) VALUES (@curso_id,
            @req);
        END
        SET @i = @i + 1;
    END
    FETCH NEXT FROM car_cursor INTO @cod_car, @nom_car;
END
CLOSE car_cursor; DEALLOCATE car_cursor;
PRINT '750 Cursos y sus requisitos creados.';
GO

SET NOCOUNT ON;
DECLARE @s INT = 1;
WHILE @s <= 1500
BEGIN
    DECLARE @id_c CHAR(6) = (SELECT TOP 1 id FROM curso ORDER BY NEWID());
    DECLARE @prefijo CHAR(2) = LEFT(@id_c, 2);

    -- Buscar un profesor que sea de la carrera del curso
    DECLARE @prof CHAR(9) = (SELECT TOP 1 dni FROM profesor WHERE id_carrera = @prefijo
    ORDER BY NEWID());

    -- Si no hay profesor de esa carrera (raro), elegir uno al azar
    IF @prof IS NULL SET @prof = (SELECT TOP 1 dni FROM profesor ORDER BY NEWID());

    DECLARE @y INT = (ABS(CHECKSUM(NEWID())) % 7) + 2020;
    DECLARE @sem INT = (ABS(CHECKSUM(NEWID())) % 2) + 1;

    INSERT INTO seccion (id_curso, dni_profesor, modalidad, cupos, horario,
    fecha_inicio, fecha_fin)
    VALUES (
        @id_c, @prof,

```

```

        (SELECT TOP 1 val FROM (VALUES ('Virtual'), ('Presencial'), ('Bimodal')) AS
M(val) ORDER BY NEWID()),
        (ABS(CHECKSUM(NEWID())) % 25) + 15,
        (SELECT TOP 1 d FROM (VALUES ('Lun-Jue'), ('Mar-Vie'), ('Mie-Sab')) AS D(d) ORDER
BY NEWID()) + ' ' + (SELECT TOP 1 h FROM (VALUES ('08:00'), ('13:00'), ('18:00')) AS H(h)
ORDER BY NEWID()),
        CASE WHEN @sem=1 THEN CAST(@y AS VARCHAR)+'-02-01' ELSE CAST(@y AS
VARCHAR)+'-08-01' END,
        CASE WHEN @sem=1 THEN CAST(@y AS VARCHAR)+'-06-15' ELSE CAST(@y AS
VARCHAR)+'-12-15' END
    );
    SET @s = @s + 1;
END
PRINT '1,500 Secciones creadas (especialidad profesor/curso vinculada).';
GO

SET NOCOUNT ON;
IF OBJECT_ID('tempdb..#Pool') IS NOT NULL DROP TABLE #Pool;

-- 1. Crear pool de candidatos válidos (Estudiante -> Seccion de su misma carrera)
SELECT
    s.id_seccion, e.dni, s.cupos,
    ROW_NUMBER() OVER (PARTITION BY s.id_seccion ORDER BY NEWID()) as FilaCupo
INTO #Pool
FROM seccion s
JOIN estudiante e ON e.id_carrera = LEFT(s.id_curso, 2) -- Misma carrera
WHERE e.fecha_ingreso <= s.fecha_inicio; -- Validar fecha de ingreso

-- 2. Insertar respetando cupos
INSERT INTO matricula (id_seccion, dni_estudiante, nota_final)
SELECT TOP 20000 id_seccion, dni, (ABS(CHECKSUM(NEWID())) % 61) + 40
FROM #Pool
WHERE FilaCupo <= cupos
ORDER BY NEWID();

PRINT 'Matrícula de 20,000 registros completada.';

```

Consultas

```

/*
EJERCICIOS DE CONSULTAS SQL - BASE DE DATOS: MATRICULA
*/
USE matricula;
GO

-- 1. SELECT, TOP, DISTINCT, ORDER BY:
-- Obtener los primeros 10 nombres únicos de los estudiantes, ordenados alfabéticamente.
SELECT DISTINCT
    TOP 10 nombre
FROM estudiante
ORDER BY nombre
GO

```

```

-- 2. WHERE, LIKE, AND, OR:
-- Seleccionar nombre, primer apellido y teléfono de estudiantes cuyo nombre empiece con
'A' o 'J'
-- y cuyo primer apellido NO termine con 'Z'.
SELECT
    nombre,
    primer_apellido,
    telefono
FROM estudiante
WHERE
    nombre LIKE 'A%' OR nombre LIKE 'J%'
    AND primer_apellido NOT LIKE '%Z'
GO

-- 3. BETWEEN, IN, NOT:
-- Mostrar dni_profesor, tipo_contrato y salario de los contratos con salario entre
250000 y 450000,
-- pero que NO sean de tipo 'cátedra' ni 'por horas'.
SELECT
    dni_profesor,
    tipo_contrato,
    salario
FROM contrato
WHERE
    salario BETWEEN 250000 AND 450000
    AND tipo_contrato NOT IN ('cátedra', 'por horas')
ORDER BY tipo_contrato, dni_profesor
GO

-- 4. IS NULL, IS NOT NULL:
-- Listar los nombres y apellidos de profesores que tienen registrado su segundo
apellido
-- pero cuya fecha_fin en el contrato es NULL (contratos vigentes).
SELECT
    nombre,
    primer_apellido,
    segundo_apellido
FROM profesor pf
INNER JOIN contrato c ON pf.dni = c.dni_profesor
WHERE fecha_fin IS NULL
ORDER BY nombre, primer_apellido, segundo_apellido
GO

-- 5. COUNT, GROUP BY, HAVING:
-- Mostrar el ID de la sección y cuántos estudiantes tiene matriculados.
-- Filtrar para ver solo las secciones con más de 2 estudiantes.
SELECT
    id_seccion,
    COUNT(dni_estudiante) AS cantidad_estudiantes
FROM matricula
GROUP BY id_seccion
HAVING COUNT(dni_estudiante) > 2
ORDER BY cantidad_estudiantes DESC
GO

```

```

-- 6. SUM, AVG, MIN, MAX:
-- Calcular el salario total, el promedio, el mínimo y el máximo de todos los
-- contratos con estado 'activo'.
SELECT
    SUM(salario) AS inversion_salarial,
    AVG(salario) AS salario_promedio,
    MIN(salario) AS salario_minimo,
    MAX(salario) AS salario_maximo
FROM contrato
WHERE estado = 'activo'
GO

-- 7. INNER JOIN:
-- Mostrar el nombre del estudiante, el nombre del curso y la nota final obtenida.
SELECT
    es.nombre,
    cu.nombre,
    nota_final
FROM matricula ma
INNER JOIN estudiante es ON es.dni = ma.dni_estudiante
INNER JOIN seccion se ON se.id_seccion = ma.id_seccion
INNER JOIN curso cu ON cu.id = se.id_curso
ORDER BY nota_final DESC
GO

-- 8. LEFT JOIN e IS NULL:
-- Listar todos los estudiantes (nombre y apellido) y su ID de sección,
-- incluyendo aquellos que aún no se han matriculado en ninguna sección (estos saldrán
-- con NULL).
SELECT
    nombre,
    primer_apellido,
    segundo_apellido,
    id_seccion
FROM estudiante es
LEFT JOIN matricula ma ON dni = dni_estudiante
GO

-- 9. RIGHT JOIN:
-- Mostrar todos los cursos (id y nombre) y la cantidad de secciones que tienen creadas.
-- Asegurarse de que se muestren los cursos aunque no tengan secciones asignadas aún.
SELECT
    cu.id,
    cu.nombre,
    COUNT(se.id_seccion)
    AS secciones_creadas
FROM curso cu
RIGHT JOIN seccion se ON se.id_curso = cu.id
GROUP BY cu.id, cu.nombre
GO

-- 10. SUBCONSULTA (Operador >):
-- Encontrar el nombre y apellidos de los estudiantes que obtuvieron una nota_final
-- mayor al promedio general de todas las notas de la institución.
SELECT DISTINCT
    nombre,

```

```

        primer_apellido,
        segundo_apellido
FROM matricula ma
INNER JOIN estudiante es
ON es.dni = ma.dni_estudiante
WHERE nota_final > (
    SELECT AVG(nota_final)
    FROM matricula)
GO

-- 11. SUBCONSULTA (Cláusula IN):
-- Listar los nombres completos de los profesores que dictan el curso con ID 'AD4045'
(matemáticas).
SELECT DISTINCT
    nombre,
    primer_apellido,
    segundo_apellido
FROM profesor pr
WHERE dni IN (
    SELECT dni_profesor
    FROM seccion
    WHERE id_curso = 'AD4045')
GO

-- 12. JOIN COMPLEJO Y LIKE:
-- Mostrar el nombre del profesor y el nombre de los cursos que imparte,
-- pero solo para cursos cuyo nombre contenga la palabra 'Administración'.
SELECT
    pr.nombre,
    cu.nombre
FROM profesor pr
INNER JOIN seccion se ON se.dni_profesor = pr.dni
INNER JOIN curso cu ON cu.id = se.id_curso
WHERE cu.nombre LIKE '%Administración%'
GO

-- 13. COUNT DISTINCT y GROUP BY:
-- Contar cuántas modalidades diferentes de clase imparte cada profesor (dni_profesor y
cantidad).
SELECT DISTINCT
    dni_profesor,
    COUNT(DISTINCT(modalidad))
    AS modalidades_impartidas
FROM seccion
WHERE dni_profesor IS NOT NULL
GROUP BY dni_profesor
ORDER BY modalidades_impartidas DESC
GO

-- 14. SUBCONSULTA CORRELACIONADA:
-- Listar los cursos que tienen créditos mayores al promedio de créditos de todos los
cursos.
SELECT
    id,
    nombre,
    credits

```

```

FROM curso
WHERE credits > (
    SELECT AVG(credits)
    FROM curso)
ORDER BY credits DESC
GO

-- 15. BETWEEN y FECHAS:
-- Mostrar los contratos que iniciaron entre el '2017-01-01' y el '2017-12-31'.
SELECT *
FROM contrato
WHERE fecha_inicio
    BETWEEN '2017-01-01' AND '2017-12-31'
GO

-- 16. NOT IN y SUBCONSULTA:
-- Mostrar los cursos que actualmente NO tienen ninguna sección abierta (no están en la
tabla seccion).
SELECT *
FROM curso
WHERE id NOT IN (
    SELECT
        id_curso
    FROM seccion)
ORDER BY credits DESC
GO

-- 17. HAVING y MÚLTIPLES JOINS:
-- Mostrar el nombre de los cursos que tienen un promedio de notas superior a 80.
SELECT
    cu.nombre,
    AVG(nota_final)
    AS notas_promedio
FROM curso cu
INNER JOIN seccion se ON se.id_curso = cu.id
INNER JOIN matricula ma ON ma.id_seccion = se.id_seccion
GROUP BY cu.nombre
HAVING AVG(nota_final) > 80
ORDER BY notas_promedio DESC
GO

-- 18. TOP y ORDER BY (Ranking):
-- Obtener los 10 estudiantes con el promedio de notas más altas de la institución,
-- mostrando su nombre y su promedio.
SELECT TOP 10
    dni_estudiante,
    AVG(nota_final) promedio
FROM matricula
GROUP BY dni_estudiante
ORDER BY promedio DESC
GO

-- 19. CASE (Opcional - Lógica condicional):
-- Mostrar el nombre del estudiante y un mensaje que diga 'Aprobado' si su nota es >= 70
-- y 'Reprobado' si es menor. (Usa la tabla matricula) y muestra el curso
correspondiente.

```

```

SELECT
    es.nombre,
    se.id_curso,
    CASE WHEN
        ma.nota_final >= 70 THEN 'Aprobado'
        ELSE 'Reprobado'
    END AS estado
FROM matricula ma
INNER JOIN estudiante es
    ON es.dni = ma.dni_estudiante
INNER JOIN seccion se
    ON se.id_seccion = ma.id_seccion
ORDER BY estado, nombre, id_curso
GO

```

Vistas

```

USE matricula;
GO
-- 1.1 CREATE VIEW:
-- Crear una vista llamada 'Vista_Detalle_Secciones' que muestre:
-- ID de sección, nombre del curso, nombre del profesor y modalidad.
CREATE OR ALTER VIEW Vista_Detalle_Secciones AS
SELECT
    id_seccion,
    cu.nombre AS estudiante,
    pr.nombre AS profesor,
    modalidad
FROM seccion se
INNER JOIN curso cu ON cu.id = se.id_curso
INNER JOIN profesor pr ON pr.dni = se.dni_profesor
GO

-- 1.2. CONSULTA SOBRE VISTA:
-- Utilizando la vista 'Vista_Detalle_Secciones', filtrar solo las secciones que son
-- 'Virtuales'.
SELECT *
FROM Vista_Detalle_Secciones
WHERE modalidad = 'Virtual'
GO

-- 2.1. CREATE VIEW (Resumen Financiero):
-- Crear una vista llamada 'Vista_Nomina_Activa' que sume el salario total que la
-- institución
-- paga por cada tipo de contrato, solo para contratos 'activos'.
CREATE OR ALTER VIEW Vista_Nomina_Activa AS
SELECT
    tipo_contrato,
    SUM(salario) AS total_salarios
FROM contrato
WHERE estado = 'Activo'
GROUP BY tipo_contrato
GO

```

```
-- 2.2 CONSULTA SOBRE VISTA:
-- Utilizando la vista 'Vista_Nomina_Activa', ordenarlos de los salarios más altos a
los más bajos.
SELECT * FROM Vista_Nomina_Activa
ORDER BY total_salarios DESC
GO

-- 3.1 CREATE VIEW (Vista de cupos disponibles)
-- Crear una vista llamada 'Vista_Cupos_Disponibles' que verifique la cantidad de cupos
disponibles
-- en el año actual
CREATE OR ALTER VIEW Vista_Cupos_Disponibles AS
SELECT ma.id_seccion,
       cupos,
       cupos - COUNT(*)
       AS cupos_disponibles
FROM matricula ma
INNER JOIN seccion se
    ON se.id_seccion = ma.id_seccion
WHERE YEAR(fecha_inicio) = YEAR(GETDATE())
GROUP BY ma.id_seccion, cupos
GO

-- 2.2 CONSULTA SOBRE VISTA:
-- Utilizando la vista 'Vista_Cupos_Disponibles', con los 10 cursos con más cupos
disponibles.
SELECT TOP 10 *
FROM Vista_Cupos_Disponibles
ORDER BY cupos_disponibles DESC
```